# 基于人脸识别的考勤系统

- 服务端AttendanceServer
  - main
    - 注册自定义数据类型到Qt元对象系统
      - qRegisterMetaType<cv::Mat>("cv::Mat&");
      - qRegisterMetaType<cv::Mat>("cv::Mat");
      - qRegisterMetaType<int64_t>("int64_t");
    - 数据库初始化
      - 连接数据库
        - 通过addDatabase（QSqlDatabase）连接db数据库
      - 设置数据库名称
        - 通过setDatabaseName设置数据库名称为server.db
      - 打开数据库
        - open打开数据库
      - 创建query（QSqlQuery）用于执行SQL语句
      - 创建员工信息表
        - 创建并执行createsql语句
      - 创建考勤表
        - 创建并执行createsql语句
  - attendancewin
    - 创建套接字mserver用于监听和接收客户端连接
    - 通过listen监听端口
      - QHostAddress::Any：用于监听所有端口
    - newconnection信号+accept_client槽函数=处理客户端连接
      - newconnection信号：当有新的客户端发起连接时触发
      - accept_client槽函数：通过nextPendingConnection()获取客户端套接字msocket，readyread信号+read_data槽函数=处理客户端数据接收
        - readyRead信号：当客户端有数据到达时触发
        - read_data槽函数：从TCP套接字中读取并解析客户端发送的人脸图像数据
          - 使用QDataStream读取和解析数据包
            - 通过setvVersion设置固定Qt版本，确保数据互通
          - 确保数据完整接收，处理分块传输的情况
            - 通过bytesAvailable判断socket是否发送完整字节数据
              - 利用>>将stream当中的backsize（图像大小信息）输入到bsize（quint64)当中
              - 利用>>将stream当中的byte（实际的JPEG图像数据）输入到data（QByteArray）当中
          - 显示接收到的图像
            - 创建mmp（QPixmap）用于修改图片格式
            - 通过loadFromData将二进制数据转为jpg格式图片
            - 通过scaled将图片缩放至标签大小
            - 通过setPixmap（ui）显示图片
          - 将图像数据转换为OpenCV格式并触发人脸识别
            - 创建faceImage（Mat）用于存储解码后的人脸图像
            - 创建decode容器用于临时存储二进制图像数据，通过resize调整容器大小以容纳整个data图像数据
            - 通过memcpy（内存复制函数）将data数据复制到decode容器当中
            - 通过imdecode（OpenCV当中的函数）将decode二进制数据解码为彩色图像输出到faceImage
            - 通过emit（发射信号）打包发射query信号，将faceImage人脸图像传递给线程当中的QFaceObject对象进行处理
    - 通过setTable给model（sql模型）绑定employee员工信息表
    - 创建多线程thread（QThread）用于单独处理人脸识别计算
      - 作用：将耗时的人脸识别计算从UI主线程中分离出来，避免界面卡顿
      - 通过moveToThread将fobj（QFaceObject人脸识别对象）移动到工作线程执行
      - 通过start启动线程
    - query信号+face_query槽函数（QFaceObject类函数）=执行人脸识别计算
      - query信号：当系统接收到客户端发送的faceImage人脸图像数据后触发
      - face_query槽函数：在人脸数据库中查找最匹配的人脸
        - 将OpenCV的Mat数据转为SeetaFace引擎所需的SeetaImage格式
          - 创建simage（SeetaImmageData格式），分别将faceImage（Mat）的data、cols、rows和channels传给simage的data、width、height、channels，实现格式转化
        - 调用Query方法在数据库中查找最匹配的人脸
          - 初始化相似度变量similarity（float）
          - 执行人脸查询
            - 调用fengineptr中的Query方法在已注册的人脸库（sqlite）进行特征匹配，将返回值人脸识别唯一标识ID存储到faceid当中，并通过指针将匹配相似度值写入similarity变量
        - 估算相似度值
          - 根据调试不断输出similarity设置相似度阈值为0.7
        - 相似度判断与结果处理
          - 高于0.7，通过emit发射send_faceid信号用于将识别结果（人脸ID）发送给recv_faceid槽函数（attendancewin类函数）
          - 低于0.7，通过emit发射send_faceid信号发送-1表示识别失败
        - 返回查询结果ID，供调用者进一步处理
    - send_faceid信号（QFaceObject）+recv_faceid槽函数=接收人脸识别结果并处理考勤逻辑
      - send_faceid信号：当系统接收到服务器发送的人脸识别ID时触发
      - recv_faceid槽函数：考勤系统核心业务
        - 验证人脸识别结果
          - 如果faceid小于0，返回错误信息
        - 查询员工数据获取个人信息
          - 通过setFilter方法对model（QSqlTableModel数据库模板）进行过滤查询设置，确保只查询匹配的员工信息
          - 通过select进行精确的数据检索
        - 数据验证与考勤流程处理
          - 通过rowCount返回结果判断确保只有一条匹配记录，保证身份识别的唯一性和准确性
          - 满足上述条件后，通过record（0）返回查询到的第一条记录输出到record（QSqliteRecord）当中
          - 创建sdmsg字符串写入JSON格式的员工核心信息以及当前的时间戳
          - 创建insertSql字符串写入添加识别成功的员工ID到考勤表当中的sql语句
          - 创建query（QSqlQuery）通过exec执行insertSql当中的sql语句
            - 成功则通过write将完整员工信息和时间戳发送给客户端
            - 失败则通过write发送空内容信息给客户端，同时用lastError记录错误信息
  - registerwin
    - 重置
      - 通过clear（ui）清空输入框的数据，时间设置为当前时间
    - 添加头像
      - 通过getOpenFileName（QFileDialog类，文件对话框）选中图片路径存储在filepath（QString）当中
      - 创建mmp（Qpixmap）并从filepath路径当中加载图片，通过scaledToWidth调整mmp尺寸以适应显示区域，最后用setPixmap（ui）将调整后的图片mmp设置到headpicLb（头像标签）上
    - 打开/关闭摄像头
      - 当text按钮文本显示“打开摄像头”时，如果摄像头已经打开，则通过setText修改按钮文本为“关闭摄像头”且启动定时器事件
        - 重写timerEvent定时器事件：
          - 通过isOpened判断摄像头是否打开，打开则将摄像头的数据定期（定时器响应时间决定）传输给image（Mat）当中，如果image为空则返回
          - 创建rgbImage（Mat），通过cvtColor（OpenCV）将Image的BGR（OpenCV格式）转为RGB（Qt QImage格式）存储在rgbImage当中
          - 通过QImage构造函数将rgbImage（Mat）图像数据存储到qimage（QImage）当中，完成最后格式转化
          - 通过fromImage（QPixmap类）将qimage（QImage）转为QPixmap格式存储在mmp当中，再通过scaledToWidth调整尺寸以适应边框大小，最后setPixmap将图像mmp显示在headpicLb（显示标签）上
    - 拍照
      - 创建headfile（QString）用于保存头像文件路径，设置路径为当前目录data文件夹下，且以员工姓名的Base64编码作为文件名，避免重名
      - 通过setText将生成的headfile文件路径显示在picfileEdit（文件路径编辑框）上，供用户查看
      - 使用imwrite（OpenCV）通过headfile路径访问将捕捉到的Mat格式数据保存为JPG格式
      - 通过killTimer停止定时器，用setText更新提示词显示“打开摄像头”以提示用户可重新打开摄像头进行采集
      - 最后通过release关闭摄像头
    - 注册
      - 创建image（Mat），通过imread（OpenCV）从picfileFdit（存放图片路径信息框）中读取图片存储在image当中
      - 创建faceobj（QFaceObject）通过face_register从image图片当中获取唯一图片ID存储在faceID（int64)_t）当中
      - 创建headfile（QString）用于保存头像文件路径，设置路径为当前目录data文件夹下，且以员工姓名的Base64编码作为文件名，避免重名
      - 通过imwrite（OpenCV）将image人脸图像写入在以headfile路径下的JPG文件当中，将员工头像持久化存储，供后续使用
      - 创建model（QSqlTabModel数据库模组），通过setTable设置要操作的表名employee
      - 通过record创建record（QSqlRecord）用于记录新员工的所有信息
      - 通过setValue依次将用户输入的信息text填充到记录当中，除了性别是通过isChecked检测单选按钮状态来设置的
      - 通过insertRecord将record记录插入到数据库表格当中且将返回值保存在ret（bool）当中
      - 通过判断ret是否为真，为真则设置information（QMessageBox）显示“注册成功”的消息提示框并且通过submitAll将所有修改提交到数据库当中，否则设置information（QMessageBox）显示“注册失败”的消息提示框
  - seletwin
    - new一个model（QSqlTableModel）数据库模型用于数据查询和显示
    - 查询
      - 通过isChecked分别检测empRb（员工单选按钮）和attRb（考勤单选按钮）如果empRb按下，则通过setTable设置为员工表；如果arrRb按下，则通过setTable设置为考勤表
      - 通过select执行查询
      - 通过setModel将查询结果设置到tableView（表格视图，Qt自带控件用于显示数据库数据）中显示
- 客户端FaceAttendance
  - faceattendannce
    - 创建cap（VideoCapture）摄像头用于捕获实时视频帧
    - 通过open（0）打开第一个摄像头
    - 创建flag用于状态控制和连续人脸检测计数
    - 通过startTimer启动定时器事件，每隔100ms调用timerEvent（）函数进行视频帧处理和人脸检测（重写定时器事件）
      - 重写timerEvent定时器事件：
        - 定时从摄像头捕获视频帧
          - 创建srcImage（Mat）用于采集图像数据
          - 通过grab从cap摄像头尝试获取下一帧图像数据
            - 成功则通过read从摄像头读取视频帧到srcImage当中
        - 对图像进行人脸识别
          - 通过resize先将srcImage图片设与显示窗口一样大
          - 创建faceRects矩形容器用于存储检测到的人脸区域坐标
          - 通过detectMultiScale级联分类器进行多尺度人脸检测并且将坐标传给faceRects
          - 通过size是否大于0（人脸矩形框数量>0）且flag是否大于等于0，即是否检测到人脸
            - 条件成立则通过at（0）从faceRects获取第一个检测到的人脸矩形框输出给rect（Rect，OpenCV中的矩形类）
            - 通过move（ui）传递rect的x和y坐标使headpicLb（人脸检测框）保持一起移动
            - 判断flag是否大于2
              - 条件成立则创建buf（uchar容器）向量缓冲区用于存储编码后的JPEG图像数据（JPEG 和 JPG 本质是同一图像格式）
              - 通过imencode（OpenCV函数）将srcImage（Mat）解码为jpg格式存储与buf当中
              - 通过QByteArray构造函数将OpenCV的字节数据buf转为Qt可处理的字节数组存储在byte（QByteArray）当中
              - 通过size将byte的数据大小存储在backsize（quint64）当中，用于记录实际字节数，便于服务端验证数据完整性
              - 创建sendData（QByteArray）用于存储和操作二进制数据
              - 通过QDataStream构造函数使用Qt数据流序列化sendData图像数据以确保不同系统间的数据兼容性
              - 通过setVersion设置Qt版本，确保数据流互通
              - 通过<<先后写入backsize（图像信息大小）和byte（图像数据），便于服务端先验证数据大小，再读取对应大小的图像数据
              - 通过网络套接字write发送sendData图形数据（序列化后）
              - 使flag值为-2
                - 确保成功连续检测到至少3帧人脸，才发送图像给服务器，这种设计避免了误检和不必要的网络传输
              - 条件不成立则flag++
          - 当size等于0时（没有检测到人脸），则通过move将headpicLb（人脸检测框）移动到中心位置
          - 如果srcImage的数据为空，则直接返回
          - 通过cvtColor用COLOR_BGR2RGB把srcImage（Mat）数据（BGR）转Qt里面的QImage（RGB）
          - 通过AImage构造函数将srcImageOpenCV的Mat格式转为Qt的QImage格式且存储在image当中
          - 通过fromImage将image的QImage格式转为QPixmap格式存储在mmp（QPixmap）当中
          - 通过setPixmap展示mmp图像内容到videoLb标签上
        - 处理检测到的人脸，当连续检测到同一人脸多次时，将图像发送到服务器
        - 将捕获的图像转换格式并且显示在UI界面上
    - 创建cascade（CascadeClassifier）Haar级联分类器用于人脸检测
    - 通过load将分类器文件导入到cascade当中，用于加载OpenCV预训练的人脸检测Haar特征分类器
    - 创建套接字msocket用于与服务器通信
    - 通过start启动定时器mtimer，每5s连接一次
    - disconnected信号+Start_connect槽函数=网络断开后一直重连
      - disconnected信号：当网络断开时触发
      - Start_connect槽函数：启动定时器事件
        - timeout定时器信号+Timer_connect槽函数=不断尝试连接服务器
          - timeout定时器信号：定时器mtimer每5s触发
          - Timer_connect槽函数：通过connectToHost让客户端主动连接指定 IP 和端口的 TCP 服务器
    - connected信号+Stop_connect槽函数=连接成功后停止重连
      - connected信号：网络成功连接时触发
      - Stop_connect槽函数：停止定时器事件
    - readyread信号+recv_data槽函数=处理服务端数据接收
      - recv_data槽函数：接收处理服务端数据
        - 通过套接字readAll读取服务端发来的数据存储到array（QByteArray）当中
        - 创建err（QJsonParseError）用于捕获错误信息
        - 通过fromJson（QJsonDocument类函数）解析array数据并且存储在doc（QJsonDocument）当中，当解析失败时返回报错信息
        - 通过object将 doc的QJsonDocument 转换为具体的 QJsonObject，方便读取 JSON 中的具体字段
        - 通过value从JSON对象中提取各个字段的值存储在各个不同的字符串数组当中
        - 通过setText显示成员信息内容，通过setStyleSheet样式（QSS）来显示图片
        - 通过show显示widgetLb（认证成功标签）
    - 通过hide（ui）隐藏widgetLb（认证成功图标）显示